ACTIVIDAD

Dosier de Actividades del Tema: Utilización avanzada de clases

Desarrollo de Aplicaciones Web/ Desarrollo de Aplicaciones Multiplataforma/Administración de Sistemas Informáticos y Redes

Programación

Linkia FP

Se debe entregar un zip que contenga todos los archivos. java que has creado. Para poder aprobar un ejercicio, éste debe poder ejecutarse sin errores.

EJERCICIO 1: Define una clase base llamada "Asignatura" con los siguientes atributos privados. Crea los setter y getters, así como el constructor:

- Nombre (String)
- Créditos (int)

```java
package DAW_M03_T9_ACT9;

/**
 *
 * @author cristiancantero
 */
public class Asignatura {
    private String nombre;
    private int creditos;


    public Asignatura() {

    }

    public Asignatura(String nombre, int creditos) {
        this.nombre = nombre;
        this.creditos = creditos;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getCreditos() {
        return creditos;
    }

    public void setCreditos(int creditos) {
        this.creditos = creditos;
    }
```

Siguiéndo el enunciado. Después de crear los atributos ==> *ctrl+i getters/setters y constructores creados.*

EJERCICIO 2: Crea dos clases heredadas llamadas "Programacion" y "BaseDatos" que extiendan la clase "Asignatura". Ambas clases deben tener los siguientes atributos privados, así como los getters, setters y constructor:

Para la clase "Programacion":

- Lenguaje (String)
- Nivel (String)

```
private String lenguaje;
private String nivel;

public Programacion() {
}

public Programacion(String lenguaje, String nivel) {
    this.lenguaje = lenguaje;
    this.nivel = nivel;
}

public Programacion(String lenguaje, String nivel, String nombre, int creditos) {
    super(nombre, creditos);
    this.lenguaje = lenguaje;
    this.nivel = nivel;
}

public String getLenguaje() {
    return lenguaje;
}

public void setLenguaje(String lenguaje) {
    this.lenguaje = lenguaje;
}

public String getNivel() {
    return nivel;
}

public void setNivel(String nivel) {
    this.nivel = nivel;
}
```

Misma operación ==> *ctrl+i getters/setters y constructores creados.*

Para la clase "BaseDatos":

- Sistema (String)
- Nivel (String)

```java
package DAW_M03_T9_AC19;

/**
 *
 * @author cristiancantero
 */
public class BaseDatos extends Asignatura {

    private String sistema;
    private String nivel;

    public BaseDatos() {
    }

    public BaseDatos(String sistema, String nivel) {
        this.sistema = sistema;
        this.nivel = nivel;
    }

    public BaseDatos(String sistema, String nivel, String nombre, int creditos) {
        super(nombre, creditos);
        this.sistema = sistema;
        this.nivel = nivel;
    }

    public String getSistema() {
        return sistema;
    }

    public void setSistema(String sistema) {
        this.sistema = sistema;
    }

    public String getNivel() {
        return nivel;
    }

    public void setNivel(String nivel) {
        this.nivel = nivel;
    }
```

Misma operación ==> *ctrl+i getters/setters y constructores creados.*

Linkia FP

EJERCICIO 3: Implementa el método abstracto "calcularPrecioTotal" en cada clase hija para calcular el precio total de la asignatura. El cálculo del precio total puede ser ficticio y basado en los créditos de la asignatura. Por ejemplo: precioTotal = número de créditos * 120;

En este apartado me encontré con la primera piedra en el camino. Primero cree una interface creyendo que sería la manera. Al definir los métodos calcularPrecioTotal y mostrarInformación, pensé que la manera seríaque luego tanto Programacion y BaseDatos la implementaran. En un principio me pareció coherente porque las interfaces te obligan a implementar métodos en las clases hijas. Pero el principio del ejercicio ya dice *"Implementa el método abstracto en cada clase hija".* Por la cual cosa, para que exista un método abstracto debe estar declarado en una clase abstracta. La idea clave de esto es:

- Cuando existe una relación de herencia y se quiere obligar a las clases hijas a implementar ciertos métodos, la solución adecuada es utilizar una clase abstracta. No una interfaz. (Chat gpt).

```java
package DAW_M03_T9_ACT9;

/**
 *
 * @author cristiancantero
 */
public abstract class Asignatura {
    private String nombre;
    private int creditos;


    public Asignatura() {

    }

    public Asignatura(String nombre, int creditos) {
        this.nombre = nombre;
        this.creditos = creditos;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getCreditos() {
        return creditos;
    }

    public void setCreditos(int creditos) {
        this.creditos = creditos;
    }

    public abstract int calcularPrecioTotal();
    public abstract void mostrarInformacion();

}
```

Así que nuestra clase abstracta Asignatura se convierte en una clase abstracta.

CalcularPrecioTotal() devolverá un int. Luego llamaremos a la función dentro de mostrarInformacion() para imprimir ese resultado.

Sin en cambio, mostrarInformacion() es void porque no nos devuelve nada. Solo muestra por pantalla los datos que le solicitamos.

Ahora vemos la implementación.

EJERCICIO 4: Implementa el método abstracto "mostrarInformacion" en cada clase hija para mostrar la información de la asignatura, incluyendo nombre, créditos y los atributos específicos de cada clase.

```java
    @Override
    public int calcularPrecioTotal(){
        int resultado = getCreditos() * 120;
        return resultado;
    }



    @Override
    public void mostrarInformacion(){
        System.out.println("Asignatura: " + getNombre());
        System.out.println("Creditos: " + getCreditos());
        System.out.println("Lenguaje: " + getLenguaje());
        System.out.println("Nivel: " + getNivel());
        System.out.println(calcularPrecioTotal());

    }

}
```

Esta es la clase Progamacion.

Aquí descubri que @Override es un marcador extraordinario. Si tocas algo del método que elimina su "autenticidad" automaticamente se ilumina en color rojo diciendote -ey, este método ya no es sobre escrito, no está mal pero no estas sobre escribiendo-

calcularPrecioTotal() está implementado de la misma forma tanto en Programacion como en BaseDatos. Soy consciente de que la lógica podría haber estado más pulida pero así se quedó. La razón real por la cual ldejo así la sintaxis es porque es una firma personal. En un mundo de IA que yo haya picado una función de esta manera deja mi impronta en este proyecto como alguien que se ha interesado en picar todas sus funciones.. El método refactorizado sería así:

*return getCreditos() * 120;*

Total, no vamos a utillizar la variable resultado para nada, sino el método. Tengo muy claro y entendido como sería este método refactorizado de esta forma.

```java
    @Override
    public void mostrarInformacion(){
        System.out.println("Asignatura: " + getNombre());
        System.out.println("Creditos: " + getCreditos());
        System.out.println("Sistema: " + getSistema());
        System.out.println("Nivel: " + getNivel());
        System.out.println(calcularPrecioTotal());

    }
```

Método abstracto sobre escrito en BaseDatos.

EJERCICIO 5: En la clase principal (llamada "Main"), crea un array estático de tipo "Asignatura" con tamaño suficiente para almacenar objetos de ambas clases hijas.

En la clase Main para crear este array estático lo haré con esta línea de código:

```
Asignatura [] asignatura = new Asignatura [20];
```

Le asigno 20 huecos de forma aleatoria dentro de un rango razonable. Para lo que ha sido ej ejercicio podría haberle asignado 2.

EJERCICIO 6: Crea instancias de las clases "Programacion" y "BaseDatos" y almacénalas en el array estático.

```
Programacion p1 = new Programacion("Java","Básico", "Programacion", 6);
BaseDatos bd1 = new BaseDatos("MySQL","Básico","Bases de datos", 4);
```

Instancias creadas con datos ya dentro del objeto.

```
asignatura[0]= p1;
asignatura[1]= bd1;
```

Y su almacenamiento posterior.

EJERCICIO 7: Recorre el array estático y llama a los métodos "mostrarInformacion" y "calcularPrecioTotal" de cada objeto para mostrar la información y el precio total de cada asignatura.

Para recorrer el array utilizaré un bucle for que recorrerá asignatura. Dentro del bucle usaré un condicional que en cada iteración comprobará si el hueco está vacío. Si no lo está llamará a mostrarInformacion() que mostrará por pantalla ambos métodos sobre escritos en Programacion y BaseDatos. Cada uno con su comportamiento pero heredado de la clase abstracta. Este es el bucle:

```
for(int i = 0; i < asignatura.length; i ++){
    if(asignatura[i]!=null){
        asignatura[i].mostrarInformacion();
        System.out.println("Precio total: " + asignatura[i].calcularPrecioTotal());
        System.out.println("----------------------------");
    }
```

BLOQUE DEL OUTPUT.

EJEMPLO DE RESULTADO

```
Asignatura: Programación 1
Créditos: 4
Lenguaje: Java
Nivel: Básico
Precio total: 400.0

Asignatura: Bases de Datos
Créditos: 5
Sistema: MySQL
Nivel: Avanzado
Precio total: 600.0
```

Este es el ejemplo requerido por el ejercicio. A continuacion pongo mi Output.

```
run:
Asignatura: Programacion
Creditos: 6
Lenguaje: Java
Nivel: Básico
720
Precio total: 720
-----------------------------
Asignatura: Bases de datos
Creditos: 4
Sistema: MySQL
Nivel: Básico
480
Precio total: 480
-----------------------------
BUILD SUCCESSFUL (total time: 0 seconds)
```

Si,estudiar es caro 😂

CONCLUSIONES FINALES.

He notado mucha más soltura a la hora de crear las clases y los constructores con respecto a los dos ejercicios anteriores.

He interiorizado bien extends e implements. He comprendido por fin el porque la interfaz es una herramienta que arregla la herencia multiple en Java.

Me he vuelto a pelear mucho con el diseño y he necesitado ayuda de chat GPT para resolverlo y sumarlo a mi experiencia. Para saber cuando usar la IA me puse un tope. Si en una hora (dos pomodoros) no resolvía el problema pediría ayuda. Y así fue.

He tenido que recurrir a documentación para recordar como se instanciaban objetos y se guardaban de forma "primigenia" para probarlos en una clase main. Llevo tanto tiempo haciendo menus y legando esa responsabilidad al usuario, que algo tan sencillo como poner los datos entre los parentesis del objeto creado y asignarle manualmente el hueco, se me había borrado completamente de la cabeza. Me vino genial refrescarlo.

Tiempo estimado de trabajo: 6 horas.

BIBLIOGRAFIA CONSULTADA:
Documentación: Imprimibles Linkia. Las ilustraciones me sirvieron mucho.

Herramientas de aprendizaje: Chat GPT Plus.

PD para Gema:

Dentro de la carpeta del proyecto hay un README.md. Es una tonteria pero he tratado dedejar un log de como heido trabajando el proyecto. En que momentos me he estancado y como he corregido las cosas. Está escrito muy punky así que no lo evalues porfa. La idea es que en un futuro yo pueda mirar ahí y ver como me las apañaba. Me lo he pasado muy bien haciendo estos trabajos. Gracias.